SSL/TLS Complete Guide

A comprehensive guide to understanding SSL/TLS protocols and certificate installation.

Table of Contents

1. Overview
2. Cryptography Fundamentals
3. x509 Certificates and Keys
4. Security through Certificates
5. Cipher Suites
6. TLS/SSL Handshake
7. TLS Defenses
8. TLS Attacks & Vulnerabilities
9. What's New in TLS 1.3
10. TLS 1.3 Under the Hood
11. TLS 1.3 Extensions

Module 1 - TLS/SSL Overview

What is SSL? What is TLS?

- **SSL (Secure Sockets Layer)**: Original protocol developed by Netscape (versions 1.0, 2.0, 3.0)
- **TLS (Transport Layer Security)**: Successor to SSL, more secure and actively maintained
- Both provide encryption, authentication, and data integrity for internet communications

How SSL/TLS Protects Your Data

Protection	Description
Confidentiality	Data encrypted so only intended recipient can read it

Protection	Description
Integrity	Data cannot be modified without detection
Authentication	Verifies identity of communicating parties

Anti-Replay and Non-Repudiation

- **Anti-Replay**: Prevents attackers from capturing and resending valid data
- **Non-Repudiation**: Sender cannot deny sending the message

Key Players

1. **Certificate Authorities (CAs)**: Issue and manage digital certificates
2. **Web Servers**: Host SSL/TLS certificates
3. **Clients/Browsers**: Validate certificates and establish secure connections
4. **Registration Authorities**: Verify identity before certificate issuance

TLS/SSL Versions

Version	Status	Notes
SSL 1.0	Never released	Security flaws
SSL 2.0	Deprecated	Vulnerable
SSL 3.0	Deprecated	POODLE vulnerability
TLS 1.0	Deprecated	Should not use
TLS 1.1	Deprecated	Should not use
TLS 1.2	Current	Widely supported
TLS 1.3	Current	Most secure, recommended

Module 2 - Cryptography

Hashing

- One-way function producing fixed-size output (digest/hash)
- Common algorithms: SHA-256, SHA-384, SHA-512
- **Avoid**: MD5, SHA-1 (compromised)

```
# Generate SHA-256 hash
echo -n "Hello World" | openssl dgst -sha256
```

Data Integrity

- Hash functions verify data hasn't been modified
- HMAC (Hash-based Message Authentication Code) adds secret key

Encryption Types

Symmetric Encryption

- Same key for encryption and decryption
- Fast, efficient for bulk data
- Algorithms: AES-128, AES-256, ChaCha20

```
# Encrypt file with AES-256
openssl enc -aes-256-cbc -salt -in file.txt -out file.enc -k password
```

Asymmetric Encryption

- Key pair: Public key (encrypt) + Private key (decrypt)
- Slower, used for key exchange
- Algorithms: RSA, ECDSA

Public Key Infrastructure (PKI)

```
┌──────────────────────────────────────────────────────────────┐
│                    Root CA (Trust Anchor)                    │
│                            │                                 │
│              ┌─────────────┴─────────────┐                   │
│              ▼                           ▼                   │
│      Intermediate CA            Intermediate CA              │
│              │                           │                   │
│         ┌────┴────┐                 ┌────┴────┐              │
│         ▼         ▼                 ▼         ▼              │
│    End-Entity  End-Entity    End-Entity  End-Entity          │
│    Certificate Certificate   Certificate Certificate         │
└──────────────────────────────────────────────────────────────┘
```

Key Exchange Algorithms

Algorithm	Type	Forward Secrecy
RSA	Asymmetric	No
DH (Diffie-Hellman)	Key Exchange	No
DHE (Ephemeral DH)	Key Exchange	Yes
ECDHE	Key Exchange	Yes

Digital Signature Algorithm (DSA)

- Proves authenticity and integrity
- Uses private key to sign, public key to verify
- Modern variant: ECDSA (Elliptic Curve DSA)

Module 3 - x509 Certificates and Keys

What is in a Certificate?

```
┌──────────────────────────────────────────┐
│           X.509 Certificate              │
├──────────────────────────────────────────┤
│ Version: 3                               │
│ Serial Number: 01:23:45:67:89            │
│ Signature Algorithm: SHA256withRSA       │
│ Issuer: CN=Example CA                    │
│ Validity:                                │
│   Not Before: Jan 1 00:00:00 2024        │
│   Not After: Dec 31 23:59:59 2024        │
│ Subject: CN=example.com                  │
│ Subject Public Key Info:                 │
│   Algorithm: RSA                         │
│   Public Key: (2048 bit)                 │
│ Extensions:                              │
│   Subject Alternative Name               │
│   Key Usage                              │
│   Extended Key Usage                     │
│ Signature: [digital signature]           │
└──────────────────────────────────────────┘
```

LAB: Inspecting a Certificate

```
# View certificate details
openssl x509 -in certificate.crt -text -noout

# View certificate of a website
openssl s_client -connect example.com:443 -servername example.com | openssl x509 -text

# Check certificate expiration
openssl x509 -in certificate.crt -noout -enddate
```

Certificate Extensions

Extension	Purpose
Subject Alternative Name (SAN)	Additional domain names
Key Usage	Allowed cryptographic operations
Extended Key Usage	Specific purposes (server auth, client auth)
Basic Constraints	CA:TRUE/FALSE, path length
CRL Distribution Points	Where to check revocation
Authority Information Access	OCSP responder URL

Private Key

```
# Generate RSA private key
openssl genrsa -out private.key 2048

# Generate EC private key
openssl ecparam -genkey -name prime256v1 -out private.key

# View private key details
openssl rsa -in private.key -text -noout
```

LAB: Matching Certificates to Private Keys

```
# Extract modulus from certificate
openssl x509 -noout -modulus -in certificate.crt | openssl md5

# Extract modulus from private key
openssl rsa -noout -modulus -in private.key | openssl md5

# If both MD5 hashes match, the key pair is valid
```

Certificate Signing Request (CSR)

```
# Generate CSR with new private key
openssl req -new -newkey rsa:2048 -nodes -keyout domain.key -out domain.csr

# Generate CSR from existing private key
openssl req -new -key private.key -out domain.csr

# View CSR contents
openssl req -in domain.csr -text -noout
```

File Formats

Format	Extension	Description
PEM	.pem, .crt, .cer, .key	Base64 encoded, most common
DER	.der, .cer	Binary format
PKCS#7	.p7b, .p7c	Certificate chain, no private key
PKCS#12	.pfx, .p12	Certificate + private key bundle

LAB: File Conversions

```
# PEM to DER
openssl x509 -in cert.pem -outform DER -out cert.der

# DER to PEM
openssl x509 -in cert.der -inform DER -outform PEM -out cert.pem

# PEM to PKCS#12
openssl pkcs12 -export -out cert.pfx -inkey private.key -in cert.pem -certfile chain.pe

# PKCS#12 to PEM
openssl pkcs12 -in cert.pfx -out cert.pem -nodes
```

LAB: Creating a Certificate Authority

```
# Step 1: Generate CA private key
openssl genrsa -out ca.key 4096

# Step 2: Create self-signed CA certificate
openssl req -new -x509 -days 3650 -key ca.key -out ca.crt \
  -subj "/C=US/ST=State/L=City/O=Organization/CN=My Root CA"

# Step 3: Generate server private key
openssl genrsa -out server.key 2048

# Step 4: Create CSR for server
openssl req -new -key server.key -out server.csr \
  -subj "/C=US/ST=State/L=City/O=Organization/CN=example.com"

# Step 5: Sign server certificate with CA
openssl x509 -req -days 365 -in server.csr -CA ca.crt -CAkey ca.key \
  -CAcreateserial -out server.crt
```

Module 4 - Security through Certificates

Certificate Validation Process

```
┌──────────────────────────────────────────────────────────────┐
│                  Certificate Validation                      │
├──────────────────────────────────────────────────────────────┤
│ 1. Check certificate signature                               │
│ 2. Verify certificate chain to trusted root                  │
│ 3. Check validity period (not expired)                       │
│ 4. Verify domain name matches (CN or SAN)                    │
│ 5. Check revocation status (CRL/OCSP)                        │
│ 6. Verify key usage extensions                               │
└──────────────────────────────────────────────────────────────┘
```

Certificate Chains

```
Root CA Certificate (Self-signed, trusted by browsers)
        │
        ▼
Intermediate CA Certificate (Signed by Root)
        │
        ▼
End-Entity Certificate (Your domain certificate, signed by Intermediate)
```

LAB: Certificate Chains

```
# View certificate chain
openssl s_client -connect example.com:443 -showcerts

# Verify certificate chain
openssl verify -CAfile chain.pem certificate.crt
```

Basic Constraints

- **CA:TRUE** - Certificate can sign other certificates
- **CA:FALSE** - End-entity certificate only
- **pathLenConstraint** - How many CAs can exist below this one

Certificate Types

Type	Validation	Time	Cost
DV (Domain Validation)	Domain control only	Minutes	$
OV (Organization Validation)	Organization verified	Days	$$
EV (Extended Validation)	Extensive verification	Weeks	$$$

Certificate Revocation

Two methods to check if certificate is revoked:

CRL (Certificate Revocation List)

```
# Download and check CRL
openssl crl -in crl.pem -text -noout
```

OCSP (Online Certificate Status Protocol)

```
# Query OCSP responder
openssl ocsp -issuer issuer.crt -cert server.crt \
  -url http://ocsp.example.com -resp_text
```

Module 5 - Cipher Suites

Cipher Suite Components

```
TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384
|    |     |        |   |   |    |
|    |     |        |   |   |    └── Hash/MAC Algorithm
|    |     |        |   |   └──────── Block Size
|    |     |        |   └──────────── Mode of Operation
|    |     |        └──────────────── Encryption Algorithm
|    |     └──────────────────────── Authentication
|    └──────────────────────────────── Key Exchange
└──────────────────────────────────── Protocol
```

Key Exchange Algorithms

Algorithm	Forward Secrecy	Recommendation
RSA	No	Avoid
DHE	Yes	Accept
ECDHE	Yes	Prefer

Encryption Algorithms

Algorithm	Key Size	Mode	Recommendation
AES-128-GCM	128-bit	AEAD	Prefer
AES-256-GCM	256-bit	AEAD	Prefer
ChaCha20-Poly1305	256-bit	AEAD	Prefer
AES-CBC	128/256-bit	Block	Accept
3DES	168-bit	Block	Avoid
RC4	Variable	Stream	Avoid

LAB: Cipher Suite Enumeration

```
# List supported cipher suites on a server
nmap --script ssl-enum-ciphers -p 443 example.com

# Test specific cipher suite
openssl s_client -connect example.com:443 -cipher ECDHE-RSA-AES256-GCM-SHA384

# List all available cipher suites
openssl ciphers -v 'ALL'
```

Module 6 - TLS/SSL Handshake

TLS 1.2 Handshake Flow

```
Client                                                Server
   |                                                     |
   |------------- ClientHello ------------------------->|
   |         (supported versions, cipher suites,         |
   |          random, extensions)                        |
   |                                                     |
   |<------------ ServerHello --------------------------|
   |         (selected version, cipher suite,            |
   |          random)                                    |
   |                                                     |
   |<------------ Certificate --------------------------|
   |                                                     |
   |<------------ ServerKeyExchange --------------------|
   |         (DH parameters, signature)                  |
   |                                                     |
   |<------------ ServerHelloDone ----------------------|
   |                                                     |
   |------------- ClientKeyExchange --------------------->|
   |         (client's DH public value)                  |
   |                                                     |
   |------------- ChangeCipherSpec ---------------------->|
   |                                                     |
   |------------- Finished ----------------------------->|
   |                                                     |
   |<------------ ChangeCipherSpec ----------------------|
   |                                                     |
   |<------------ Finished ------------------------------|
   |                                                     |
   |<============ Encrypted Data ========================>|
```

LAB: Inspecting TLS Handshake in Wireshark

```
# Capture TLS handshake
tcpdump -i any -w tls_capture.pcap port 443

# Or use tshark
tshark -i any -f "port 443" -w tls_capture.pcap
```

Wireshark filter: `ssl.handshake`

Session Resumption

Two methods:

1. **Session IDs**: Server stores session state
2. **Session Tickets**: Client stores encrypted session state

TLS Extensions

Extension	Purpose
SNI (Server Name Indication)	Multiple certificates on one IP
OCSP Stapling	Server provides OCSP response
Session Tickets	Stateless session resumption
ALPN	Application protocol negotiation

LAB: Decrypting TLS in Wireshark

```
# Set SSLKEYLOGFILE environment variable
export SSLKEYLOGFILE=/tmp/sslkeys.log

# Run your browser or curl
curl https://example.com

# In Wireshark: Edit → Preferences → Protocols → TLS
# Set "(Pre)-Master-Secret log filename" to /tmp/sslkeys.log
```

Module 7 - TLS Defenses

HTTP Strict Transport Security (HSTS)

Forces browsers to use HTTPS only.

```
Strict-Transport-Security: max-age=31536000; includeSubDomains; preload
```

Certificate Authority Authorization (CAA)

DNS record specifying which CAs can issue certificates for your domain.

```
# Check CAA records
dig CAA example.com

# Example CAA record
example.com. IN CAA 0 issue "letsencrypt.org"
```

Certificate Transparency (CT)

Public logs of all issued certificates for auditing.

- Certificates must be logged to CT logs
- Browsers verify SCT (Signed Certificate Timestamp)
- Detect rogue/misissued certificates

Module 8 - TLS Attacks & Vulnerabilities

Major Vulnerabilities

Attack	Vulnerability	Mitigation
POODLE	SSL 3.0	Disable SSL 3.0
BEAST	TLS 1.0 CBC	Use TLS 1.1+
CRIME	TLS Compression	Disable compression

Attack	Vulnerability	Mitigation
Heartbleed	OpenSSL bug	Patch OpenSSL
DROWN	SSLv2	Disable SSLv2
ROBOT	RSA key exchange	Use ECDHE
Logjam	Weak DH params	Use 2048-bit+ DH

Session Renegotiation Attack

Secure renegotiation extension (RFC 5746) fixes this vulnerability.

```
# Test for secure renegotiation
openssl s_client -connect example.com:443 -status
```

Module 9 - What's New in TLS 1.3

Key Differences from TLS 1.2

Feature	TLS 1.2	TLS 1.3
Handshake RTT	2-RTT	1-RTT (0-RTT resumption)
Key Exchange	RSA, DHE, ECDHE	ECDHE, DHE only
Cipher Suites	Many	5 recommended
Encryption	CBC, GCM	AEAD only
Forward Secrecy	Optional	Mandatory
Renegotiation	Supported	Removed

TLS 1.3 Cipher Suites

```
TLS_AES_256_GCM_SHA384
TLS_AES_128_GCM_SHA256
TLS_CHACHA20_POLY1305_SHA256
TLS_AES_128_CCM_SHA256
TLS_AES_128_CCM_8_SHA256
```

TLS 1.3 Handshake (1-RTT)

```
Client                                                  Server
   |                                                       |
   |──────────────── ClientHello ──────────────────────────▶|
   |          (+ key_share extension)                      |
   |                                                       |
   |◀──────────────── ServerHello ─────────────────────────|
   |◀──────────────── {EncryptedExtensions} ───────────────|
   |◀──────────────── {Certificate} ───────────────────────|
   |◀──────────────── {CertificateVerify} ─────────────────|
   |◀──────────────── {Finished} ──────────────────────────|
   |                                                       |
   |──────────────── {Finished} ───────────────────────────▶|
   |                                                       |
   |◀═══════════════ Encrypted Data ═══════════════════════▶|

{} = Encrypted
```

Module 10 - TLS 1.3 Under the Hood

TLS 1.3 Key Schedule

```
PSK (if resuming) ──────┐
                        ▼
                ┌─────────────────┐
      0 ──────► │ HKDF-Extract    │──────► Early Secret
                └─────────────────┘
                        │
                        ▼
                ┌─────────────────┐
ECDHE ──────► │ HKDF-Extract    │──────► Handshake Secret
                └─────────────────┘
                        │
                        ▼
                ┌─────────────────┐
      0 ──────► │ HKDF-Extract    │──────► Master Secret
                └─────────────────┘
```

0-RTT (Zero Round Trip Time)

- Send application data with first message
- Risk: Replay attacks possible
- Use only for idempotent requests

```
# Check if server supports 0-RTT
openssl s_client -connect example.com:443 -tls1_3 -early_data file.txt
```

Session Resumption in TLS 1.3

- Uses PSK (Pre-Shared Key) mechanism
- No session IDs
- NewSessionTicket sent after handshake

Module 11 - TLS 1.3 Extensions

Key Extensions

Extension	Purpose
supported_versions	Negotiate TLS version
key_share	Send key exchange data upfront
signature_algorithms	Supported signature algorithms
psk_key_exchange_modes	PSK with or without DHE
early_data	Enable 0-RTT
cookie	Stateless retry

ALPN (Application Layer Protocol Negotiation)

Negotiate application protocol during TLS handshake.

```
# Test ALPN
openssl s_client -connect example.com:443 -alpn h2,http/1.1
```

Post-Handshake Authentication (PHA)

Request client certificate after handshake completion.

SSL Certificate Installation - Step by Step

Step 1: Generate Private Key

```
# RSA 2048-bit key
openssl genrsa -out yourdomain.key 2048

# Or ECDSA key (recommended)
openssl ecparam -genkey -name prime256v1 -out yourdomain.key
```

Step 2: Create CSR

```
openssl req -new -key yourdomain.key -out yourdomain.csr \
  -subj "/C=US/ST=State/L=City/O=Organization/CN=yourdomain.com"
```

Step 3: Submit CSR to Certificate Authority

Options:

- **Let's Encrypt** (Free, automated)
- **DigiCert, Comodo, GoDaddy** (Paid)
- **AWS ACM, Cloudflare** (Free with their services)

Step 4: Complete Domain Validation

Method	Description
Email	Respond to email sent to admin@yourdomain.com
DNS	Add TXT record to DNS
HTTP	Upload file to /.well-known/pki-validation/

Step 5: Install Certificate

Apache

```
<VirtualHost *:443>
    SSLEngine on
    SSLCertificateFile /path/to/yourdomain.crt
    SSLCertificateKeyFile /path/to/yourdomain.key
    SSLCertificateChainFile /path/to/chain.crt
</VirtualHost>
```

Nginx

```
server {
    listen 443 ssl;
    ssl_certificate /path/to/fullchain.crt;
    ssl_certificate_key /path/to/yourdomain.key;
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256;
}
```

Step 6: Verify Installation

```
# Test SSL configuration
openssl s_client -connect yourdomain.com:443 -servername yourdomain.com

# Online tools
# - https://www.ssllabs.com/ssltest/
# - https://www.sslshopper.com/ssl-checker.html
```

Let's Encrypt with Certbot

```
# Install certbot
sudo apt install certbot python3-certbot-nginx

# Obtain and install certificate
sudo certbot --nginx -d yourdomain.com -d www.yourdomain.com

# Auto-renewal (runs twice daily by default)
sudo systemctl status certbot.timer
```

Quick Reference Commands

```
# Generate key + CSR in one command
openssl req -new -newkey rsa:2048 -nodes -keyout domain.key -out domain.csr

# Check certificate expiration
openssl x509 -in cert.crt -noout -dates

# Check if certificate matches private key
diff <(openssl x509 -noout -modulus -in cert.crt) <(openssl rsa -noout -modulus -in key

# Test SSL connection
openssl s_client -connect example.com:443 -servername example.com

# Check certificate chain
openssl s_client -connect example.com:443 -showcerts

# View certificate details
openssl x509 -in cert.crt -text -noout

# Check TLS version support
openssl s_client -connect example.com:443 -tls1_3
```

Resources

- Mozilla SSL Configuration Generator
- SSL Labs Server Test
- Let's Encrypt
- OWASP TLS Cheat Sheet